



WEEKLY INTERNSHIP PROGRESS REPORT

Module: Week 2 – Database & CRUD Operations in Laravel

Submitted By: Muneeb ur Rehman

Role: Full-Stack Web Developer Intern

Stack/Focus: PHP, Laravel, MySQL, Bootstrap 5

Report Date: 11-Aug-2026

Submitted To: POF IT Dept

1. Executive Summary & Objective

The primary objective for Week 2 was to transition from foundational PHP concepts to dynamic, database-driven application development using the Laravel framework. The core deliverable was to architect and develop a fully functional **Student Management System** utilizing the MVC (Model-View-Controller) design pattern. This included configuring relational database schemas, implementing Eloquent ORM, establishing RESTful resource routing, and building a professional UI with strict server-side data validation.

2. Task Breakdown: What We Did & How We Did It

Day 6: Database Configuration & Schema Migrations

- **What was achieved:** Established the connection between the Laravel backend and the local MySQL database and created the foundational data structures.
- **Execution (How):**
 - Updated the .env configuration file to map to the student_management MySQL database.
 - Generated a migration file (php artisan make:migration create_students_table) to programmatically define the database schema.
 - Structured the table to include id, name, email (unique constraint), phone (nullable), course, and Laravel's default timestamps.
 - Executed the migration using php artisan migrate to construct the tables in the database.

Day 7: Eloquent Models & Data Protection

- **What was achieved:** Created the data access layer to interact with the newly created database tables using object-oriented syntax.
- **Execution (How):** * Generated the Student model alongside its controller using php artisan make:model Student -c.
 - Configured the Model to protect against mass-assignment vulnerabilities by explicitly defining the \$fillable array (name, email, phone, course), ensuring only authorized data points can be injected via HTTP requests.

Day 8 & 9: Complete CRUD Operations & Form Validation

- **What was achieved:** Engineered the core business logic to Create, Read, Update, and Delete student records.
- **Execution (How):**
 - **Routing:** Implemented `Route::resource('students', StudentController::class)` in `web.php` to automatically generate all 7 standard RESTful routes.
 - **Controller Logic:** Programmed the `StudentController` methods (`index`, `create`, `store`, `edit`, `update`, `destroy`).
 - **Validation:** Implemented strict server-side request validation in the `store` and `update` methods. Ensured emails remained unique to the database (ignoring the current user's email during the update process using `'unique:students,email,' . $student->id`).

Day 10: Professional UI & Advanced Blade Templating

- **What was achieved:** Wrapped the raw backend logic in a clean, responsive, and user-friendly frontend interface.
- **Execution (How):**
 - Built a master layout file (`app.blade.php`) utilizing **Bootstrap 5** via CDN for rapid, professional styling.
 - Utilized Blade template inheritance (`@extends` and `@section`) to inject dynamic content (`Create`, `Edit`, and `Index` views) into the master layout.
 - Integrated CSRF protection (`@csrf`) into all HTML forms to secure the application against Cross-Site Request Forgery attacks.

3. Technicalities & Advanced Implementations

Beyond the standard requirements, several production-grade features were engineered to ensure the application is scalable and performant.

1. **Dynamic Conditional Querying:** Instead of fetching all database records blindly, the application checks for user search inputs. We utilized Laravel's Eloquent `when()` method, which executes search logic *only* if a search string is present in the HTTP request.
2. **Partial String Matching:** Implemented SQL LIKE operators within the Eloquent query (`('%{$search}%')`) allowing users to filter the database by partial names, emails, or courses simultaneously.

3. **Server-Side Pagination:** To prevent memory overflow and optimize rendering times as the database grows, `Student::all()` was replaced with `Student::paginate(5)`. This delegates the chunking of data directly to the SQL layer.

4. Challenges Encountered & Solutions Handled

During development, a few technical edge cases arose which required specific handling:

- **Challenge 1: Search State Loss on Pagination.** * *Issue:* When a user searched for a term yielding multiple pages, clicking "Page 2" would drop the search parameter from the URL, resetting the table.
 - *Resolution:* Handled this by chaining the `appends(request()->query())` method onto the paginator in the controller. This forces Laravel to persist all active URL search parameters across pagination links.
- **Challenge 2: Broken UI on Empty Search Results.**
 - *Issue:* Searching for a non-existent student resulted in an empty, broken HTML table layout.
 - *Resolution:* Replaced the standard `@foreach` Blade loop with the `@forelse` directive. This allowed for the graceful rendering of a fallback UI state (`<td colspan="5">No students found</td>`) when the data array returned empty.
- **Challenge 3: CSS Framework Conflicts.**
 - *Issue:* Laravel defaults to Tailwind CSS for its built-in pagination components, causing our Bootstrap 5 UI to render massive, unstyled SVGs.
 - *Resolution:* Overrode the default framework compiler by importing the `Paginator` class in the `AppServiceProvider.php` file and invoking `Paginator::useBootstrapFive()` in the `boot()` method.

5. User Interface (UI) Showcase

Figure 1: Main Dashboard (Read & Delete)

Student Management

All Students

Search by name, email, or course...

Search

Add New Student

ID	Name	Email	Course	Actions
1	Muneeb Ur Rehman	awanzada1289@gmail.com	math	EditDelete
2	Muhammad Adeel	awanzada189@gmail.com	eng	EditDelete
3	subhan	awanzada129@gmail.com	gho	EditDelete
4	safeer	awanzada128@gmail.com	compt	EditDelete
5	naser	awanzad1289@gmail.com	islamya	EditDelete

↩️ 🔍

idnameemailphonecoursecreated_atupdated_at

☐

EditCopyDelete

1Muneeb Ur Rehmanawanzada1289@gmail.com03499881955math2026-08-08 16:26:572026-08-08 16:26:57

☐

EditCopyDelete

2Muhammad Adeelawanzada189@gmail.com03499881955eng2026-08-08 16:32:162026-08-08 16:32:16

☐

EditCopyDelete

3subhanawanzada129@gmail.com03499881955gho2026-08-08 16:32:302026-08-08 16:32:30

☐

EditCopyDelete

4safeerawanzada128@gmail.com03499881955compt2026-08-08 16:32:492026-08-08 16:32:49

☐

EditCopyDelete

5naserawanzad1289@gmail.com03499881955islamya2026-08-08 16:33:072026-08-08 16:33:07

⬆️

☐ Check all

With selected:

EditCopyDeleteExport

Figure 2: Student Registration (Create & Validate)

Register New Student

Name

Email

Phone

Course

Save Student

Cancel

Figure 3: Record Modification (Update)

Course	Actions
math	Edit Delete

Edit Student

Name

Muneeb Ur Rehman

Email

Phone

03499881955

Course

math

Update Student

Cancel

Please fill in this field.

6. Conclusion

Week 2 successfully bridged the gap between basic PHP syntax and modern framework architecture. By completing the Student Management System, a solid foundation in the MVC design pattern, object-relational mapping, secure routing, and template rendering has been established, preparing the workflow for Intermediate Laravel concepts in Week 3.